Lecture 8 Exercise: Git

Course: DM571 / DM889 Software Engineering
Time: about 35 minutes, in 5 parts
You need: Git, Python 3, a terminal, a partner, and a GitHub account each.
Files: the exercise file comes from GitHub. `git/campusbike.py` is only a spare copy for the one-laptop flow.

Case: CampusBike

> **CampusBike** is the made-up bike rental for the university from Lecture 7. Students take a bike at a station on campus, ride, and bring it back.
>
> You and your partner work on the same CampusBike file, in one shared repository on GitHub. You make conflicts on purpose and solve them, you use branches, a pull request, and at the end you read the history you made.

Work in pairs: **Person A** and **Person B**, each on your **own laptop**, in the **terminal**. Do not edit files on the GitHub website: then the conflicts never reach your laptop, and you miss the point.

Part 1 - Set up: fork and clone (5 min)

Both: check that Git works, and tell Git who you are. Use your own name and the email of your GitHub account. If you skip this, `git commit` stops with "Author identity unknown".

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

> **Python:** this sheet says `python3`. On Windows, type `py` instead.
>
> **Windows only:** run this once, so Windows line endings do not turn every line into a change:
> `git config --global core.autocrlf true`

The starting file is in a public course repository:
`https://github.com/Oliver1703dk/campusbike-git`

Person A:

1. Open the course repository and click **Fork**. Keep the default name (`campusbike-git`) and owner (you). Click **Create fork**.
2. You are now on **your fork**: the address starts with `github.com/A-USERNAME/`. Go to **Settings, Collaborators, Add people**, type Person B's GitHub username, and click **Add B-USERNAME to campusbike-git**.

Person B:

1. Accept the invitation: check your email, or open `https://github.com/A-USERNAME/campusbike-git/invitations` (or `github.com/notifications`). You can only push after you accept.

Both:

1. Clone **A's fork**, not the course repository. Copy the **HTTPS** URL from the green **Code** button on A's fork:

```
git clone https://github.com/A-USERNAME/campusbike-git.git
cd campusbike-git
```

2. Check that `campusbike.py` is there, and run it: `python3 campusbike.py`.

Push to A's fork only.

- Push says `403` or "Permission ... denied"? Run `git remote -v`. If it shows the course repository, you cloned the wrong one. Fix it:
  `git remote set-url origin https://github.com/A-USERNAME/campusbike-git.git`
- **No pull request in Parts 1-3.** Never open one to the course repository. In Part 4 you open pull requests, and the base repository must be **A's fork**.

Before your first push:

Logging in to GitHub from the terminal. The first push asks you to log in. **Your GitHub password does not work here.**

- **A browser window opens** (usual on Windows): log in there. Done.
- **The terminal asks for Username and Password** (usual on Mac and Linux): type your GitHub username. As the password, paste a **token**. To make one on github.com: your picture, **Settings, Developer settings, Personal access tokens, Tokens (classic), Generate new token (classic)**. Tick **repo**, click **Generate**, copy the token. Nothing shows while you paste. That is normal: press Enter.
- Use a **classic** token. A fine-grained token cannot push to your partner's repository.

Part 2 - Your first conflict, on main (8 min)

Both of you: open `campusbike.py` **inside your `campusbike-git` folder** in an editor, and change the **same line**. Save.

Person A	Person B
`MAX_RENTAL_HOURS = 4`	`MAX_RENTAL_HOURS = 2`

Step 1 - Person A first: commit and push.

```
git add campusbike.py
git commit -m "Set max rental time to 4 hours"
git push
```

Step 2 - Then Person B: commit and push.

```
git add campusbike.py
git commit -m "Set max rental time to 2 hours"
git push
```

The push is **rejected**: `! [rejected] main -> main (fetch first)`. GitHub has A's commit, and B does not have it yet.

Step 3 - Person B: get A's commit and merge it.

```
git pull --no-rebase
```

Git says `CONFLICT (content): Merge conflict in campusbike.py`. Open the file. You see something like:

```
<<<<<<< HEAD
MAX_RENTAL_HOURS = 2
=======
MAX_RENTAL_HOURS = 4
>>>>>>> bb86d3893a205ab42a6a4e4c79e07c18b65d0f14
```

The part above `=======` is B's version (`HEAD`). The part below is A's version, from GitHub.

Step 4 - Agree on one value together. Person B edits the file: keep **one** line with the value you agree on, and delete the three marker lines (`<<<<<<<`, `=======`, `>>>>>>>`). Save. Then check that the file still runs:

```
python3 campusbike.py
```

Step 5 - Person B: finish the merge and push. Put your agreed value in the message.

```
git add campusbike.py
git commit -m "Merge: agree on 4 hours"
git push
```

Step 6 - Person A: get the merge: `git pull --no-rebase`. Now you both have the same file.

Tips

- **Commit before you pull.** With changes that are not committed, Git stops: "Your local changes ... would be overwritten by merge".
- Use `git pull --no-rebase`. Plain `git pull` may stop with "Need to specify how to reconcile divergent branches".
- Always use `git commit -m "..."`. Without `-m`, Git opens an editor, often vim. To get out of vim: press `Esc`, type `:wq`, press Enter.
- Git does **not** check that the markers are gone. That is why you run the file before `git add`.
- Lost in the middle of a merge? `git merge --abort` takes you back to before the pull.
- Push says `403` or "Permission denied": Person B has not accepted the invitation yet, you cloned the course repository instead of A's fork (see Part 1), or the computer is logged in as someone else.

Part 3 - Branches that merge cleanly (7 min)

This time you each work on **your own branch** and change a **different** line.

	Person A	Person B
Branch	`price`	`late-fee`
Change	`PRICE_PER_HOUR = 12`	`LATE_FEE = 40`

Step 1 - Both, at the same time: make your branch, change your line, save, commit, and push the branch.

Person A:

```
git switch -c price
```

Now edit the line in `campusbike.py` and save. Then:

```
git add campusbike.py
git commit -m "Raise price to 12 DKK per hour"
git push -u origin price
```

Person B:

```
git switch -c late-fee
```

Now edit the line in `campusbike.py` and save. Then:

```
git add campusbike.py
git commit -m "Lower late fee to 40 DKK"
git push -u origin late-fee
```

The first push of a new branch needs `-u origin` and the branch name. After that, plain `git push` works.

Step 2 - Person A, after B has pushed: merge both branches into `main` and push.

```
git switch main
git pull
git merge price
git merge origin/late-fee -m "Merge late-fee"
python3 campusbike.py
git push
```

- `git pull` also downloads B's branch. On your computer it is called `origin/late-fee`.
- `git merge price` says `Fast-forward`: `main` had no new commits, so Git just moves `main` forward.
- `git merge origin/late-fee` says `Merge made by the 'ort' strategy.` **No conflict:** the two changes are on different lines, so Git combines them by itself. Check: the file has both 12 and 40.

Step 3 - Person B: get the new `main`.

```
git switch main
git pull
```

Part 4 - A pull request with a conflict (10 min)

You do all editing, branching, committing, pushing and conflict fixing on your own computer, in the terminal and your editor. Only the pull request itself happens on github.com: you open it (Compare & pull request, base repository = A's fork) and click Merge.

Both of you change the **same line** in `rental_price()`: `price = hours * PRICE_PER_HOUR`.

	Person A	Person B
Branch	`start-fee`	`first-hour-free`
New line	`price = 5 + hours * PRICE_PER_HOUR`	`price = (hours - 1) * PRICE_PER_HOUR`
Idea	5 DKK start fee	First hour free

Keep the 4 spaces in front of the line.

Step 1 - Both: start from the new `main`, make your branch, change the line, save, and commit.

```
git switch main
git pull
git switch -c start-fee
```

Now edit the line in `campusbike.py` and save. Then:

```
git add campusbike.py
git commit -m "Add a start fee of 5 DKK"
```

Person B uses `first-hour-free` and the message `"Make the first hour free"`.

Step 2 - Person A: push the branch, then go to **A's fork** on github.com.

```
git push -u origin start-fee
```

1. Click **Compare & pull request**.
2. Check the top line. **base repository** must be **A's fork** (`A-USERNAME/campusbike-git`), **base** `main`, **compare** `start-fee`. GitHub picks the course repository first: change **base repository** to A's fork.
3. Click **Create pull request**.
4. **Person A: wait until B has committed (Step 1) before you click Merge.** Then click **Merge pull request** and **Confirm merge**.

Step 3 - Person B: push your branch and open a pull request the same way (base repository = A's fork, base `main`, compare `first-hour-free`).

```
git push -u origin first-hour-free
```

Before you click **Create pull request**, GitHub may say **"Can't automatically merge"**. Create the pull request anyway, and **do not merge it yet**. On the pull request page GitHub then says **"This branch has conflicts that must be resolved"**. **Do not click Resolve conflicts.** You fix it on your computer.

Step 4 - Person B, on your computer: merge the new `main` into your branch.

```
git pull --no-rebase origin main
```

Git says `CONFLICT`. In the file:

```
<<<<<<< HEAD
    price = (hours - 1) * PRICE_PER_HOUR
=======
    price = 5 + hours * PRICE_PER_HOUR
>>>>>>> 80446f26ab86f8aae7457d15a742b277ad87f53f
```

Agree with A on one line. Keep it, delete the markers, save, and run the file. Then commit and push. Put what you agreed in the message; this example is for the start fee:

```
git add campusbike.py
git commit -m "Merge main: agree on the start fee"
git push
```

Step 5 - Person A, on github.com: reload B's pull request. It now says **"This branch has no conflicts with the base branch"**. Person A (the owner) clicks **Merge pull request** and **Confirm merge**.

Step 6 - Both: get the final `main` and run it.

```
git switch main
git pull
python3 campusbike.py
```

Part 5 - Read the history (5 min)

Both:

```
git log --oneline --graph
git blame campusbike.py
```

`git blame` shows, for every line, the commit and the person that last changed it. Long output: press Space for more, `q` to quit.

1. **Who changed which line?** Use `git blame`. Who last changed `MAX_RENTAL_HOURS`, `PRICE_PER_HOUR`, `LATE_FEE` and the `price = ...` line? In Part 2, Person B made the merge commit: whose name is on the `MAX_RENTAL_HOURS` line, and why?
2. **Where are the merges?** Find the merge commits in the graph. Which ones did GitHub make?
3. **Why are there side branches** (lines that split off and join again)? Why is there no side line for `price` from Part 3?

No GitHub account? Parts 2-4 on one laptop

No GitHub account, or the login does not work? Cloning a public repository needs no account. Branches play the two people. Nothing is pushed.

The rule: make the branch first, before you change anything. Then it starts from the old value. A branch made after a change on `main` already has the new value, and nothing conflicts.

Part 2. Clone, and make the branch right away. `main` and `student-b` both have `MAX_RENTAL_HOURS = 3`.

```
git clone https://github.com/Oliver1703dk/campusbike-git.git
cd campusbike-git
git branch student-b
```

1. On `main`: edit to `MAX_RENTAL_HOURS = 4`, save, `git add campusbike.py`, `git commit -m "Set max rental time to 4 hours"`.
2. `git switch student-b`. The file says 3 again. Edit the same line to `MAX_RENTAL_HOURS = 2`, save, `git add campusbike.py`, `git commit -m "Set max rental time to 2 hours"`.
3. `git switch main`, then `git merge student-b`: `CONFLICT`. Here `HEAD` is `main` (4) and `student-b` is 2. Fix the file as in Part 2, Step 4, then `git add campusbike.py` and `git commit -m "Merge student-b: agree on 4 hours"`.

Part 3. Make both branches first, from `main`: `git branch price` and `git branch late-fee`.

1. `git switch price`: `PRICE_PER_HOUR = 12`, save, `git add campusbike.py`, `git commit -m "Raise price to 12 DKK per hour"`.
2. `git switch late-fee`: `LATE_FEE = 40`, save, `git add campusbike.py`, `git commit -m "Lower late fee to 40 DKK"`.
3. `git switch main`, `git merge price`, then `git merge late-fee -m "Merge late-fee"`. No conflict.

Part 4. A plain merge replaces the pull request. Make both branches first, from `main`: `git branch start-fee` and `git branch first-hour-free`.

1. `git switch start-fee`: change the `price = ...` line to A's version, save, `git add campusbike.py`, `git commit -m "Add a start fee of 5 DKK"`.
2. `git switch first-hour-free`: change the same line to B's version, save, `git add campusbike.py`, `git commit -m "Make the first hour free"`.
3. `git switch main`, `git merge start-fee`, then `git merge first-hour-free`: `CONFLICT`. Fix the file, then `git add campusbike.py` and `git commit -m "Merge first-hour-free: agree on the start fee"`.

Part 5 works the same. No internet at all? `git init -b main` in a new folder, copy `git/campusbike.py` into it, `git add` and `git commit` it, then start with `git branch student-b`.